



SSFuzz: Generating syntactic and semantic seeds for RISC-V processors

Shaoqian Jin

Institute of Information Engineering, Chinese Academy of Sciences

School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China

jinshaoqian@iie.ac.cn

Liwei Chen*

Institute of Information Engineering, Chinese Academy of Sciences

School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China

chenliwei@iie.ac.cn

Yulin Li

Institute of Information Engineering, Chinese Academy of Sciences

School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China

liyulin@iie.ac.cn

Gang Shi

Institute of Information Engineering, Chinese Academy of Sciences

School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China

shigang@iie.ac.cn

ABSTRACT

As processor complexity increases, the significance of fuzz testing for identifying vulnerabilities escalates. The effectiveness of fuzz testing is closely linked to the quality of its seeds. Randomly generated fuzzing seeds are often inefficient and struggle to thoroughly explore the processor's state. In contrast, high-quality seeds improve register coverage and accelerate bug detection. In our research, we refine seed optimization through syntactic and semantic analysis. We introduce SSFuzz, a fuzzer designed to generate superior seeds by employing syntax for mutations and expansions, and by utilizing diverse semantics to deepen seed instruction logic. This approach enables a more rapid exploration of the processor's state space and facilitates the discovery of bugs more efficiently. Utilizing ANTLR4[8], we convert code into Abstract Syntax Trees (AST) for seed generation and mutation, while also augmenting seed control flow to boost instruction execution rates. By guiding the insertion of semantically rich code fragments through coverage feedback, we enhance the effectiveness of our methodology. Our experiments validate the strategy's success, demonstrating significant improvements in seed execution rates and register coverage on Rocket[1] and Boom[15] processors.

CCS CONCEPTS

• **Hardware** → **Hardware test.**

KEYWORDS

processor, fuzzing, syntactic, semantic



This work is licensed under a Creative Commons Attribution-NonCommercial International 4.0 License.

GLSVLSI '24, June 12–14, 2024, Clearwater, FL, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0605-9/24/06

<https://doi.org/10.1145/3649476.3658712>

ACM Reference Format:

Shaoqian Jin, Yulin Li, Liwei Chen, and Gang Shi. 2024. SSFuzz: Generating syntactic and semantic seeds for RISC-V processors. In *Great Lakes Symposium on VLSI 2024 (GLSVLSI '24)*, June 12–14, 2024, Clearwater, FL, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649476.3658712>

1 INTRODUCTION

As processors grow in complexity, so do security concerns. Even a minor bug in processor design can lead to significant damage, exemplified by the costly recall of defective processors due to the Pentium FDIV bug[2], which produced incorrect binary floating-point results during specific divisions, costing Intel approximately \$475 million. The RTL (Register Transfer Level) design of CPU processors, being a combinational logic of hardware circuits, can be conceptualized as a finite state machine. Identifying flaws within the CPU's RTL design necessitates an exhaustive exploration of this finite state machine's state space. For expansive processor designs, this state space is vast, necessitating the generation and execution of a large volume of seeds. Effective and high-quality seeds are crucial for thoroughly exploring the processor's state space, enhancing the efficiency of fuzz testing, and facilitating the discovery of defects and potential security vulnerabilities.

Fuzz testing's core principle is leveraging randomization to navigate the system's state space. However, given the increasing size of software and hardware systems, it is often impractical to cover all possible states within a finite timeframe. This underscores the need not just for automated detection methods, but for those that are both efficient and precise. Analogously, while a monkey typing at a computer might eventually produce a masterpiece given infinite time, relying on such random chance for breakthroughs is generally untenable. Similarly, in practical chip development, there is no luxury of infinite time for a processor to autonomously explore all state spaces. Optimizing fuzz testing seeds, therefore, involves narrowing down the input space to focus on potentially problematic states rather than expending time on common, issue-free states. Coverage-guided seed mutation[3, 10, 13, 14], a prevalent technique

in fuzz testing, aims to address this, yet most seeds currently used in processor fuzz testing overlook the importance of seed syntax and semantics. The resulting randomly generated seeds are often disorganized, making it challenging to pinpoint potential vulnerabilities.

To tackle these challenges, we introduce SSFuzz, a fuzzer designed to mutate and extend assembly instructions based on RISC-V[9] syntax while incorporating semantics into random seeds. SSFuzz, integrated into the DiFuzzRTL[4] prototype, produces seeds suitable for testing across any RISC-V processor. Recognizing that random mutation of assembly instructions can disrupt their format, we developed the first RISC-V assembly grammar description file using ANTLR4 rules. This enables SSFuzz to transform assembly instructions into abstract syntax trees with ANTLR4, allowing for expansions and mutations that preserve syntax integrity and offer finer mutation granularity.

Moreover, we observed that the execution rate of randomly generated seed instructions by DiFuzzRTL was notably low, with many instructions generated but only a few executed at any given time due to undefined random jumps and interrupt exceptions. SSFuzz introduces control flow semantics and employs a maze algorithm to establish certain control flows, altering the assembly instructions' execution logic and boosting seed execution rates. Additionally, to enhance the contextual relationship between instructions, SSFuzz guides the insertion of seeds with semantic logic blocks through mutation, incorporating POC code and assembly instructions generated by tools like Csmith[11] and Yarp[7].

Our preliminary evaluation of SSFuzz-generated seeds on Rocket and Boom processors revealed a substantial improvement in register coverage, with a 2.5-fold increase on Rocket and a 1.5-fold increase on Boom. This demonstrates the effectiveness of our approach in optimizing seed quality for processor fuzz testing, thereby enhancing the discovery and rectification of processor defects and vulnerabilities.

2 BACKGROUND

In this section, we provide a briefly background of Hardware fuzz testing and RISC-V. Finally, we will introduce the ANTLR4 parser.

2.1 Hardware fuzz testing

RFUZZ[5] is the first research to implement coverage-guided fuzzing in the hardware domain. RFuzz uses multiplexer coverage as a fuzz testing mutation metric. DifuzzRTL is an open source fuzz testing tool that provides relatively complete processor fuzz testing. DifuzzRTL instruments the firrtl code of the processor's middle layer, and then compiles it to generate Verilog code for register coverage statistics. DifuzzRTL uses the golden reference model to perform differential fuzz testing. The randomly generated test vector RISC-V assembly instructions are executed at the RTL and ISA levels respectively, and then the data in the processor memory and the data in the register are compared to find the bugs. At the same time, the register coverage ratio is used as an indicator to guide seed mutation. DifuzzRTL inserts 3 new registers in each module. The regstate register is used to record status information, the covmap register is used to record the coverage information of the current module and the covsum register is used to summarize and record

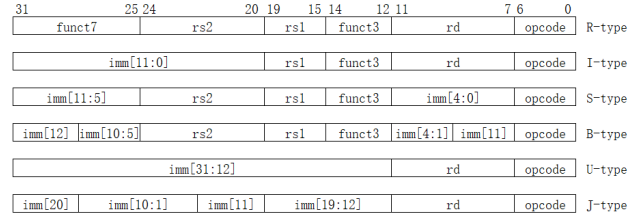


Figure 1: RISC-V instruction format

Table 1: RISC-V instruction set architecture privilege level

level	coding	name	abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

the coverage data of all modules. The value of the control register[4] is XORed with the value of the reg state register, and then written to the covmap register. If it is a new register state, then add one to the cov sum register to calculate the total control register of the CPU Coverage. TheHuzz[12] directly uses existing commercial standard tools such as Synopsys VCS to process information. However, these hardware fuzzers do not use syntax and semantics to optimize the input seeds, so this article optimizes the input seeds based on syntax and semantics.

2.2 RISC-V Instruction Set Architecture

RISC-V Instruction Set Architecture (ISA) is an open source reduced instruction set architecture that has gradually been widely used. Due to its free and open characteristics, it is welcomed by industry and academia. It is the fifth generation reduced instruction set architecture developed by the University of California, Berkeley, so it is called RISC-V. RISC-V instruction format

The RISC-V instruction set includes six formats: R, I, S, B, U, and J, with Figure 1 outlining their structures. R-type instructions perform register-to-register operations using three 5-bit register addresses, determined by a 7-bit opcode, funct7, and 3-bit funct3. I-type instructions, for short immediate data and memory access, involve two register addresses and a 12-bit immediate value. S-type instructions, indicated by a 7-bit opcode and 3-bit funct3, execute memory store operations like "sw," "sh," and "sb." B-type instructions, for conditional jumps like "beq" and "bne," use two source registers and a 12-bit immediate. U-type instructions handle long immediate data operations with a destination register and a 20-bit immediate value, as seen in "lui." J-type instructions, such as "jal," facilitate unconditional jumps.

RISC-V's architecture defines four privilege levels: U, S, H, and M, with ascending access rights. M mode, the highest level, grants access to all hardware and privileged instructions, typically for bare metal execution. S level is for operating systems, U level for applications, and H level, reserved for future virtualization improvements, indicates RISC-V's evolving support for virtualization.

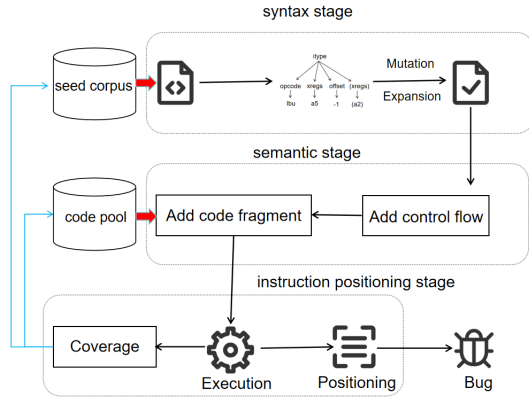


Figure 2: SSFuzz Framework

2.3 ANTLR4

ANTLR (Another Tool for Language Recognition) is a powerful tool for building language processing applications like lexical and syntax analyzers. ANTLR4, its latest version, supports multiple programming languages including Java, Python, and C#, and is popular for developing compilers, interpreters, and code generators. Key features of ANTLR4 include the formal definition of syntax using EBNF-like specifications, support for multiple target languages, efficient lexical and syntactic analysis, automatic syntax tree generation, and advanced error handling mechanisms for precise syntactic error reporting. One significant advantage of ANTLR4 is its ability to automatically generate Abstract Syntax Trees, which represent the syntax structure of source code in a tree-shaped data structure, abstracting away syntax details to focus on structure. This automatic generation of AST from syntax specifications streamlines development, enhancing both efficiency and code quality without the need for manual coding.

3 DESIGN AND IMPLEMENTATION

In this section, we will briefly introduce the design and implementation of each part of SSFuzz, including how to optimize fuzz testing seeds from the perspective of syntax and semantics.

3.1 SSFuzz overview

SSFuzz mainly solves two existing core problems: 1. How to mutate seeds efficiently without destroying the syntax format; 2. How to make seeds execute efficiently while the instructions are accompanied by semantic logic. The overall logical framework is shown in Figure 2. First, a seed is randomly selected from the fuzzing input seed library and instruction blocks are selected from it. Use the implemented RISC-V assembly syntax description rule file and use the ANTLR4 tool to convert it into an AST abstract syntax tree. Then the abstract syntax tree is traversed and each node on the abstract syntax tree is mutated, so that the syntactic structure is not destroyed. To address the low execution rate of seed instructions,

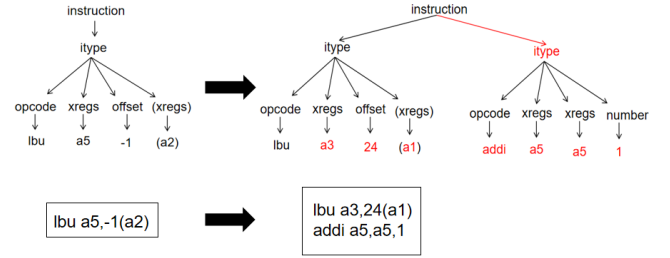


Figure 3: Using syntax to mutate and extend assembly instructions

assembly instructions within the seed are organized into basic instruction blocks without jump instructions between them. Clear jump logic is added at the end of each block to ensure sequential control flow and prevent infinite loops. By treating each block as a maze cell, a maze algorithm creates a unique path, representing the sole control flow, which renders the execution path of the assembly instructions more controlled and predictable. To improve processor control register coverage and identify potential defects or security vulnerabilities, instructions are enhanced by incorporating semantic logic from various sources, including RISC-V processor POC codes, which represent deeper processor state spaces. Additionally, compiling C/C++ code generated by Csmith and Yarpgen into assembly instructions ensures these instructions maintain contextual relationships, a feature difficult to achieve with random generation. The deterministic control flow of seeds generated by SSFuzz allows for the binary method to be used in locating inconsistent instructions, thereby streamlining the analysis and enhancing the efficiency of defect or vulnerability detection. Seeds are executed by both the processor and the Spike simulator for comparison of register and memory values. Mismatch between Spike and processors indicate potential design flaws or security issues.

3.2 syntax stage

The syntax stage primarily aims to mutate and expand RISC-V assembly instructions to enhance seed diversity. Traditional mutation methods, such as simple insertions or deletions, fail to significantly improve processor control register coverage due to minimal state space transitions and inaccessible coverage points. Random mutations risk breaking the instruction format, rendering seeds unusable and reducing fuzz testing efficiency. To address these issues, we introduce the first ANTLR4-based syntax description file for RISC-V assembly instructions, enabling the conversion of these instructions into Abstract Syntax Trees (ASTs). By mutating and expanding AST nodes, we generate more diverse seeds without compromising the fundamental syntax of RISC-V assembly, thus enhancing fuzz testing control and efficiency. Our grammar-based fuzz testing approach consists of three steps: preprocessing grammar rules and assembly instructions, mutating and expanding AST leaf nodes, and converting the AST back into assembly instruction blocks. Figure 3 effectively demonstrates this methodology.

Algorithm 1 Grammar-based seed mutation and expansion

Input: The Seed Inputs S , The Syntax G , The probability of mutation P .

Output: The New Seed S' .

```

1:  $Word \leftarrow \text{GetWordFromSeed}(S)$ 
2:  $Lexer, Parser \leftarrow \text{ProcessSyntax}(G)$ 
3:  $Tree \leftarrow \text{ConvertTree}(Lexer, Parser, Word)$ 
4: for  $node$  in  $Tree$  do
5:    $p \leftarrow \text{random}()$ 
6:   if  $p < P$  then
7:      $node \leftarrow \text{NodeMutate}(node)$ 
8:      $Tree' \leftarrow \text{GenRandomTree}()$ 
9:      $node \leftarrow \text{AddNode}(Tree')$ 
10:  end if
11: end for
12:  $Word' \leftarrow \text{ConvertWord}(Tree)$ 
13:  $S' \leftarrow \text{InsertWord}(Word')$ 

```

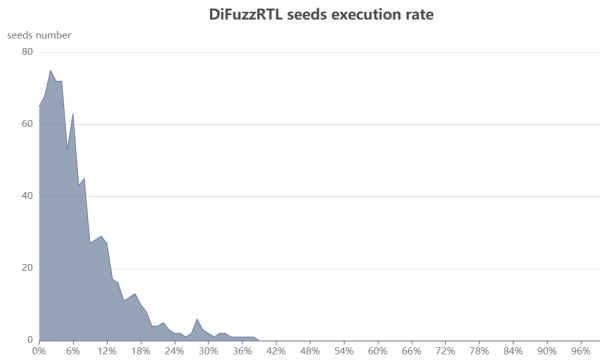


Figure 4: DiFuzzRTL seed execution rate

3.3 semantic stage

We provide a total of 2 methods to enhance the semantic logic of seeds. The first is to add a certain control flow to the seed instruction, and the second is to insert code fragments with semantic information through coverage. The execution rate of seed instructions currently used for processor fuzz testing is low. There are two main reasons. First, there are many random jump instructions. During the actual execution process, only a few instructions may be executed, and then the jump instructions appear until the end of the program. Second, it is limited by the assembly template provided by RISC-V tests. That is, once an exception occurs, the program will terminate immediately and no subsequent instructions will be executed. As shown in Figure 4, this article uses the Spike simulator to count the number of instructions actually executed for fuzz testing out of about 800 seeds randomly generated by DiFuzzRTL. The execution rate of most seeds from DiFuzzRTL is under 10%, making many instructions ineffective. Adopting Fuzzle's maze algorithm approach[6], this study introduces control flow semantics to create a unified control flow for programs, enhancing fuzz testing efficiency. We expand instruction blocks to hold 15 to 20 instructions, avoiding both the lack of logical instruction relationships from smaller blocks and the inefficiencies caused by

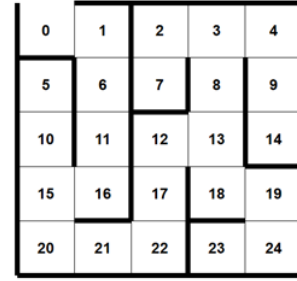


Figure 5: A maze used to represent control flow

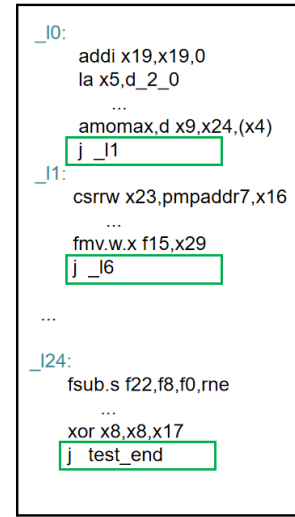


Figure 6: Add control flow to assembler

overly large blocks. By excluding jump instructions and treating each block as a cell within a maze with a single path, we ensure a streamlined and predictable control flow, optimizing fuzz testing performance. Each cell in Figure 5 is a basic code block. There is only one solution path in this maze, and there is only one entrance and exit, namely (0→1→6→11→16→15→20→21→22→17→12→13→18→19→24). This sequence is called a control flow sequence. Through this control flow sequence, the corresponding code block jump instructions are added at the end of each code block, so that the final assembly program can be completely executed according to the control flow sequence, which greatly improves the execution rate of the assembly instructions, thus improving the control Register coverage, Figure 6 shows the practical effect of adding control flow sequences in the assembler. To further enhance the semantic depth of seeds, ensuring that the instructions within the seeds maintain contextual relationships, we propose a method that leverages coverage to guide the insertion of instruction blocks. Initially, we collected Proof of Concept (POC) codes from various processors, along with C/C++ codes generated by tools like Csmith and Yarpgen, storing them as RISC-V assembly instructions. These instruction blocks, when compiled, transform into sequences of

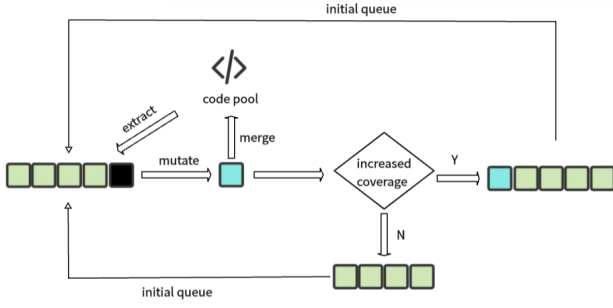


Figure 7: Coverage guides code insertion

assembly instructions with inherently strong semantics. This transformation process is notably challenging to reverse; generating a series of assembly instructions that replicate the effect of specific C code through random generation is exceedingly difficult. Inserting semantically rich code fragments typically leads the processor into a new state space. Our method aims to identify code fragments that significantly enhance processor coverage, mutating these fragments to swiftly achieve improved semantic results. Figure 7 illustrates the process of mutating code blocks guided by coverage. Starting with an initial code block selection queue of five, when a seed incorporates code fragment semantics, the foremost code in the queue is selected for mutation and then integrated into the seed. If this mutation round results in increased coverage, the mutated code fragments are recognized as new, valid semantic code blocks, subsequently re-entered into both the code pool and selection queue. Conversely, if coverage does not increase, the code fragments are omitted. Once the selection queue depletes, code blocks are randomly chosen from the code pool to replenish the queue, ensuring a continuous process of semantic enhancement.

3.4 Instruction positioning stage

After obtaining inconsistent information, we can only know that the seed can trigger the problem, but more human analysis is needed for the specific instructions that cause the problem. In order to improve the efficiency of fuzz testing and improve the ability to locate defective seed problem instructions, this paper proposes an algorithm for quickly locating inconsistent information instructions.

Algorithm2 shows the process of locating faulty instructions. First, let L represent the first instruction block in the control flow and R represent the last instruction block in the control flow. Re-execute the first instruction block. If the inconsistency error still occurs, it means that the problematic instruction occurs in the first instruction block. If no inconsistency error occurs, let M be the $(L + R)/2$ instruction block. Re-execute all instructions from the L -th instruction block to the M -th instruction block. If no inconsistency error occurs, it means that the erroneous instruction occurs between the M -th instruction block and the R -th instruction block. If an inconsistency error occurs, it means that the erroneous instruction occurs between the L -th and M -th instruction blocks. Using this recursive search, the problematic instruction block can finally be located within $O(\log n)$ time complexity. Finally, since the number of instructions between a single instruction block is small, you only

Algorithm 2 Dichotomy positioning error

Input: The Mismatch Seed Inputs S , Control Flow Sequence CFS .
Output: Block number containing the faulty instruction L .

```

1:  $L \leftarrow CFS[0]$ 
2:  $R \leftarrow CFS[-1]$ 
3:  $S' \leftarrow \text{SplitSeed}(S, L, CFS)$ 
4: if exec( $S'$ ) cause mismatch then
5:   return  $L$ 
6: end if
7: do
8:    $M \leftarrow (L + R)/2$ 
9:    $S' \leftarrow \text{splitSeed}(S, M, CFS)$ 
10:  if exec( $S'$ ) cause mismatch then
11:     $R \leftarrow M$ 
12:  else
13:     $L \leftarrow M + 1$ 
14:  end if
15: while  $L < R$ 
16: return  $L$ 

```

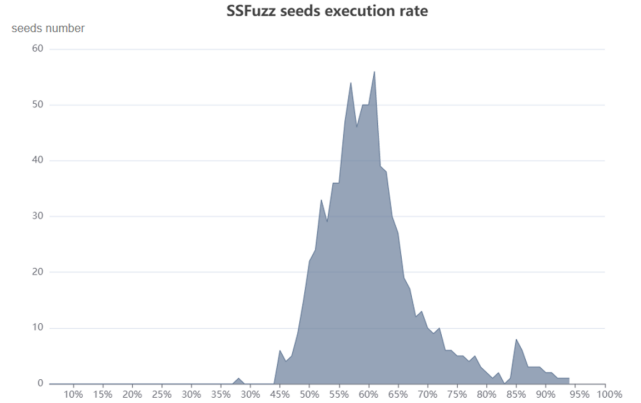


Figure 8: SSFuzz seeds execution rate

need to add each instruction to the execution to locate the specific instruction in which the inconsistent error occurred.

4 EVALUATION

4.1 Instruction execution rate

Figure 8 illustrates the improvement in seed execution rate achieved by SSFuzz through the incorporation of control flow semantic optimization. Utilizing Spike, we analyzed approximately 800 seeds generated by SSFuzz, observing that the majority of seed execution rates fall between 50% and 70%. This represents a significant enhancement compared to the execution rates of seeds produced by DiFuzzRTL. The limitation in achieving higher execution rates primarily stems from the application of the maze algorithm in generating control flow paths; not all blocks are integrated into the control flow, meaning the program's execution will not navigate through every instruction block.

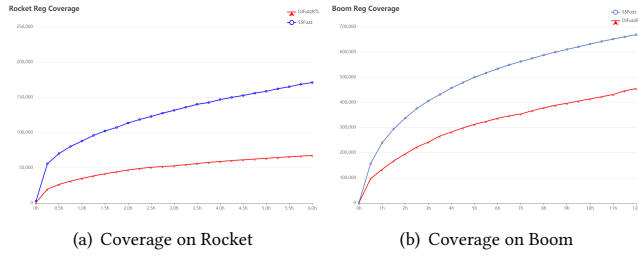


Figure 9: Coverage on Boom and Rocket

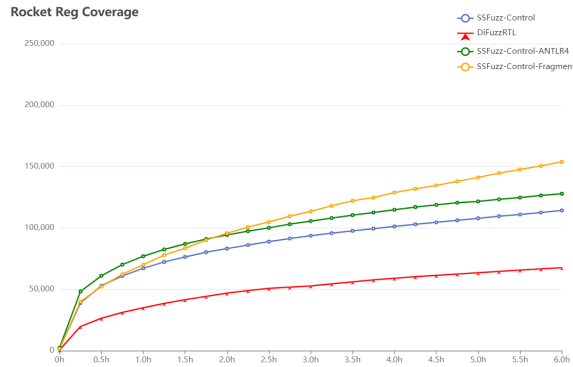


Figure 10: Coverage growth of various semantic strategies

4.2 Register coverage

First we evaluated the register coverage of SSFuzz and DiFuzzRTL generated seeds on Rocket and Boom. Figure 9 shows the comparison of the register coverage of seeds generated by SSFuzz with DiFuzzRTL on Rocket and Boom processors respectively. On Rocket, after 6 hours of running, SSFuzz has 2.5x the register coverage of DiFuzzRTL. On Boom, due to the larger scale of Boom, the original register coverage is also greater. DiFuzzRTL reached nearly 450k after 12 hours, while the seeds generated by SSFuzz reached nearly 670K, which is 1.5 times the register coverage of DiFuzzRTL.

Then we compared the register coverage growth of each syntax and semantics on the Rocket processor in Figure 10. Since the execution rate of the original seed is low, adding separate syntax and semantics is difficult to produce an effect, so the syntax and semantics will be added to the control flow semantic for experiments. The blue curve shows the coverage growth of SSFuzz after adding control flow semantics to the seed. At 6 hours, it reached 1.7 times the coverage of DiFuzzRTL. The yellow curve shows the change in coverage after adding the semantics of coverage-guided code snippet insertion on the basis of control flow semantics. After running for 6 hours, the register coverage reached an average of 2.2 times that of DiFuzzRTL. Taking 100k coverage as the standard, DiFuzzRTL needs to run for nearly 22 hours, while SSFuzz only takes about 1.5 hours to achieve 100k coverage.

Table 2: Elapsed time to reach 100K coverage

Coverage	Elapsed time				
	SSFuzz	Control	ANTLR4	Fragment	DiFuzzRTL
100K	1.5h	4.0h	2.5h	2.25	22h

5 CONCLUSION

In this work, we proposed SSFuzz, which can extend and mutate assembly instructions based on grammar, and add semantic information to fuzz testing seeds. In the experiment, the seeds generated by SSFuzz achieved certain results on Rocket and Boom. Compared with DiFuzzRTL, the register coverage rate was increased by 2.5 times on Rocket, and the register coverage rate was increased by 1.5 times on Boom.

ACKNOWLEDGMENTS

This work is partially supported by the National Natural Science Foundation of China (No. 62172407), and the Youth Innovation Promotion Association CAS.

REFERENCES

- [1] Krste Asanovic, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelewitz, et al. 2016. The rocket chip generator. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17 4* (2016), 6–2.
- [2] Intel Corporation. 1999. Pentium processor specification update.
- [3] Shai Fine and Avi Ziv. 2003. Coverage directed test generation for functional verification using bayesian networks. In *Proceedings of the 40th annual Design Automation Conference*. 286–291.
- [4] Jaewon Hur, Suhwan Song, Dongup Kwon, Eunjin Baek, Jangwoo Kim, and Byoungyoung Lee. 2021. Difuzzrtl: Differential fuzz testing to find cpu bugs. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1286–1303.
- [5] Kevin Laefer, Jack Koenig, Donggyu Kim, Jonathan Bachrach, and Koushik Sen. 2018. RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [6] Haeun Lee, Soomin Kim, and Sang Kil Cha. 2022. Fuzzle: Making a Puzzle for Fuzzers. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.
- [7] Vsevolod Livinskii, Dmitry Babokin, and John Regehr. 2020. Random testing for C and C++ compilers with YARPGen. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–25.
- [8] Terence J. Parr and Russell W. Quong. 1995. ANTLR: A predicated-LL (k) parser generator. *Software: Practice and Experience* 25, 7 (1995), 789–810.
- [9] riscv-isa sim. 2024. *RISC-V Software*. <https://github.com/riscv-software-src/riscv-isa-sim>
- [10] Giovanni Squillero. 2005. Microgp—an evolutionary assembly program generator. *Genetic programming and evolvable machines* 6 (2005), 247–263.
- [11] Kai Tamkun. 2021. Using Csmith to Find Compiler Bugs. (2021).
- [12] Aakash Tyagi, Addison Crump, Ahmad-Reza Sadeghi, Garrett Persyn, Jeyavijayan Rajendran, Patrick Jauernig, and Rahul Kande. 2022. Thehuzz: Instruction fuzzing of processors using golden-reference models for finding software-exploitable vulnerabilities. *arXiv preprint arXiv:2201.09941* (2022).
- [13] Ilya Wagner, Valeria Bertacco, and Todd Austin. 2005. StressTest: an automatic approach to test generation via activity monitors. In *Proceedings of the 42nd annual Design Automation Conference*. 783–788.
- [14] Fanchao Wang, Hanbin Zhu, Pranjay Popli, Yao Xiao, Paul Bodgan, and Shahin Nazarian. 2018. Accelerating coverage directed test generation for functional verification: A neural network-based framework. In *Proceedings of the 2018 on Great Lakes Symposium on VLSI*. 207–212.
- [15] Jerry Zhao, Ben Korpan, Abraham Gonzalez, and Krste Asanovic. 2020. Sonic-boom: The 3rd generation berkeley out-of-order machine. In *Fourth Workshop on Computer Architecture Research with RISC-V*, Vol. 5.